

APÊNDICE

Guia de Refatoração e Evolução do Código

Objetivo

Este documento tem como finalidade orientar a equipe na evolução do sistema desenvolvido na AV1 para a versão AV2.

A proposta não consiste em desenvolver um novo sistema do zero, mas sim em realizar uma refatoração no código do projeto existente, preservando as funcionalidades já implementadas e incorporando os novos conceitos estudados durante a disciplina.

O processo deverá ser realizado de forma incremental, garantindo que o sistema permaneça funcional durante todas as etapas da evolução.

Diretrizes Gerais de Desenvolvimento

Antes do início das atividades, a equipe deverá observar as seguintes recomendações:

- Manter o sistema compilando durante todo o processo de desenvolvimento.
- Realizar commits frequentes e descritivos (mensagem de indicam o que foi adicionado/moficado/removido do código).
- Testar o funcionamento do sistema após cada etapa concluída.
- Corrigir erros de compilação imediatamente após sua ocorrência.
- Evitar múltiplas alterações simultâneas sem validação intermediária.

A estratégia recomendada consiste em evoluir gradualmente o sistema, validando cada conceito antes de prosseguir para o próximo.

Etapa 1 – Compreensão da Solução Existente

Objetivo

Compreender completamente a estrutura implementada na AV1 antes do início da refatoração.

Atividades

A equipe deverá ser capaz de identificar:

- Responsabilidade de cada classe existente;

- Fluxo completo de cadastro de pacientes;
- Fluxo completo de cadastro de profissionais;
- Fluxo completo de agendamento;
- Fluxo completo de atendimento;
- Fluxo completo de pagamento;
- Utilização de construtores sobrecarregados;
- Utilização de métodos sobrecarregados.

Critério de Conclusão

Todos os membros da equipe devem ser capazes de explicar o funcionamento do sistema sem consultar o código-fonte.

Etapa 2 – Construção da Hierarquia de Pessoas

Objetivo

Aplicar os conceitos de abstração e herança.

Atividades

Criar a classe abstrata:

Pessoa

A classe deverá concentrar atributos comuns aos participantes do sistema.

Atributos sugeridos

- nome;
- cpf;
- telefone;
- dataNascimento.

Operações obrigatórias

- getters e setters;
- método abstrato `exibirResumo()`.

Resultado Esperado

Centralização dos atributos compartilhados em uma única superclasse.

Etapa 3 – Especialização da Classe Paciente

Objetivo

Transformar Paciente em uma especialização de Pessoa.

Atividades

- Aplicar herança utilizando extends Pessoa;
- Remover atributos duplicados;
- Utilizar super(...) nos construtores;
- Sobrescrever o método exibirResumo().

Resultado Esperado

Paciente funcionando como uma especialização da classe Pessoa.

Etapa 4 – Especialização da Classe Profissional

Objetivo

Transformar Profissional em uma superclasse abstrata.

Atividades

- Aplicar herança utilizando extends Pessoa;
- Tornar a classe abstrata;
- Criar método abstrato: registrarEspecifico()

Resultado Esperado

Estrutura preparada para especializações profissionais.

Etapa 5 – Construção da Hierarquia de Profissionais

Objetivo

Implementar herança especializada.

Atividades

Criar as seguintes subclasses:

- Fisioterapeuta;

- Psicologo;
- Nutricionista;
- ClinicoGeral.

Cada especialização deverá possuir:

- atributos específicos;
- sobrescrita de métodos;
- comportamento próprio para registro clínico.

Resultado Esperado

Hierarquia profissional completa e funcional.

Etapla 6 – Encapsulamento e Controle de Acesso

Objetivo

Aplicar encapsulamento em todas as entidades do sistema.

Atividades

- Tornar atributos privados;
- Criar getters e setters;
- Aplicar validações em setters críticos;
- Revisar modificadores de acesso.

Regras

Ao menos dois atributos deverão possuir validação de consistência.

Resultado Esperado

Nenhum atributo acessado diretamente fora da classe.

Etapla 7 – Interfaces e Contratos

Objetivo

Aplicar abstração através de interfaces.

Atividades

Criar as interfaces:

Agendavel

Exportavel

Responsabilidades

Agendavel:

- agendar;
- cancelar;
- remarcar.

Exportavel:

- exportarDados.

Resultado Esperado

Classes utilizando contratos formais de comportamento.

Etapa 8 – Refatoração da Hierarquia de Pagamentos

Objetivo

Aplicar abstração, polimorfismo e ligação dinâmica.

Atividades

Transformar:

Pagamento

em classe abstrata.

Criar:

- PagamentoDinheiro;
- PagamentoCartao;
- PagamentoConvenio.

Cada subclasse deverá implementar:

calcularValorFinal()

Resultado Esperado

Processamento de pagamentos baseado em polimorfismo.

Etapa 9 – Implementação dos Relacionamentos

Objetivo

Aplicar associação, agregação e composição.

Atividades

Composição

Criar: Prontuario relacionado a: Atendimento

Agregação

Criar: HorarioDisponivel relacionado a: Profissional

Associação

Criar: Convenio relacionado a: Paciente

Resultado Esperado

Modelagem correta dos relacionamentos estudados.

Etapa 10 – Modernização da Estrutura de Dados

Objetivo

Substituir estruturas fixas por coleções.

Atividades

Implementar:

List: ArrayList

Set: HashSet

Map: HashMap

Aplicações sugeridas

- Pacientes;
- Profissionais;
- Consultas;
- Atendimentos;
- Pagamentos;
- Controle de CPFs;

- Busca rápida de entidades.

Resultado Esperado

Sistema livre de arrays fixos e contadores manuais.

Etapas 11 – Exceções Personalizadas

Objetivo

Aplicar tratamento orientado a exceções.

Atividades

Criar exceções de negócio específicas.

Sugestões

- `PacienteNaoEncontradoException;`
- `PacienteInativoException;`
- `HorarioIndisponivelException;`
- `ConvenioNaoCobreException;`
- `PagamentoInvalidoException.`

Resultado Esperado

Regras de negócio tratadas por exceções especializadas.

Etapas 12 – Tratamento de Erros

Objetivo

Garantir robustez da aplicação.

Atividades

Aplicar:

- `try;`
- `catch;`
- `finally;`
- `throw;`
- `throws.`

Requisitos

- Entradas inválidas não devem encerrar a aplicação.
- Erros devem ser tratados de forma amigável ao usuário.

Resultado Esperado

Sistema resiliente a erros operacionais.

Etapa 13 – Organização da Lógica de Negócio

Objetivo

Separar responsabilidades da aplicação.

Atividades

Criar:

ClinicaServico

A classe deverá concentrar as regras de negócio do sistema.

Responsabilidades da Main

- menus;
- entrada de dados;
- exibição de resultados.

Resultado Esperado

Separação clara entre interface e lógica de negócio.

Etapa 14 – Polimorfismo, Casting e Ligação Dinâmica

Objetivo

Consolidar os conceitos avançados de orientação a objetos.

Atividades

Implementar:

Relatório Unificado, utilizando: List<Pessoa>

Relatório de Pagamentos, utilizando: List<Pagamento>

Casting Seguro, utilizando: instanceof

Resultado Esperado

Uso efetivo de polimorfismo e ligação dinâmica.

Etapa 15 – Atualização da Documentação

Objetivo

Registrar a estrutura final do sistema.

Entregáveis

Diagrama de Classes

Deve apresentar:

- herança;
- interfaces;
- associação;
- agregação;
- composição.

Mapa de Aplicação dos Conceitos

Identificando onde cada conceito foi utilizado.

Jornadas de Usuário

- Jornadas da AV1;
- Novas jornadas da AV2.

Resultado Esperado

Documentação alinhada ao código implementado.

Considerações Finais

A avaliação desta etapa não considera apenas o funcionamento do sistema, mas principalmente a qualidade da solução orientada a objetos construída pela equipe.

A implementação deverá demonstrar domínio dos conceitos estudados durante toda a disciplina, evidenciando a capacidade de modelar, organizar e evoluir sistemas utilizando os princípios da Programação Orientada a Objetos em Java.